
ECE 414/517: Reinforcement Learning

Homework 1 + Coding Assignment 1

Fall 2026

Instructor: Fei Liu

Released: Monday, September 14, 2026

Due: Thursday, Oct 8, 2026 at 11:59 p.m. ET

Purpose and learning objectives

This assignment develops the foundations used throughout the course. After completing it, you should be able to (i) formulate a sequential decision problem as an MDP; (ii) reason about horizons, discounting, and reward specification; (iii) evaluate a fixed policy and analyze Bellman operators; and (iv) implement and compare policy iteration and value iteration on a finite MDP.

Submission instructions

Submit the following two items separately on Canvas.

1. **Homework 1 (written):** one PDF containing your answers to Problems 1–5. You need to submit one PDF file including all your answers. You can produce the file however you like (e.g. LaTeX, Microsoft Word, very neat handwriting, etc) as long as it is readable. Points will be deducted if we have a hard time reading your solutions.
2. **Coding Assignment 1:** one ZIP containing the completed `coding/` directory. Do not include virtual environments, cache directories, or generated files.

Use Python 3.10 or newer and NumPy. late-work, and academic-integrity policies posted on Canvas.

Conventions for the coding assignment

States and actions are represented by zero-based integer indices. A value function has shape `(num_states,)` and a deterministic policy has shape `(num_states,)` with integer action entries. If multiple actions have the same value, choose the smallest action index (the behavior of `numpy.argmax`). Iterative methods must stop when the infinity norm of the change is at most the requested tolerance and must also respect `max_iterations`. Do not use Gymnasium or an RL library for this assignment; the goal is to implement dynamic programming directly.

Homework 1: Written Problems (100 points)

1 Effective horizon (24 points)

Consider an inventory-management MDP with states $s \in \{0, 1, \dots, 10\}$, where s is the number of items in stock. The initial state is $s = 3$. The actions are *sell* and *buy*.

- If $s > 0$, selling gives reward $+1$ and moves to $s - 1$; at $s = 0$, selling gives reward 0 and leaves the state unchanged.
 - If $s < 9$, buying gives reward 0 and moves to $s + 1$. Buying at $s = 9$ moves to terminal state $s = 10$ and gives reward $+100$.
- (a) For a finite, undiscounted horizon H , can an optimal trajectory from $s = 3$ contain both buy and sell actions? What's the differences between $H = 7$ and $H = 9$? Explain. [6 pts]
- (b) In the infinite-horizon discounted problem, can one choose $\gamma \in [0, 1)$ so that an optimal policy from $s = 3$ never reaches $s = 10$? Explain; a numerical value is not required. [6 pts]
- (c) Give one concrete pair (H, γ) for which the action selected at every state reachable from $s = 3$ by a finite-horizon optimal policy agrees with an infinite-horizon discounted optimal policy. Justify your pair. [6 pts]
- (d) Does such a γ necessarily exist for every H ? Briefly justify your answer. [6 pts]

2 Reward specification (16 points)

An autonomous-vehicle designer wants to reduce mean commute time for all drivers but uses mean vehicle speed as a simpler proxy reward. In the scene below, there is one AI-controlled car (red) and many human-controlled cars (gray). Under the proxy reward, the AI car may learn to park instead of merging onto the highway.

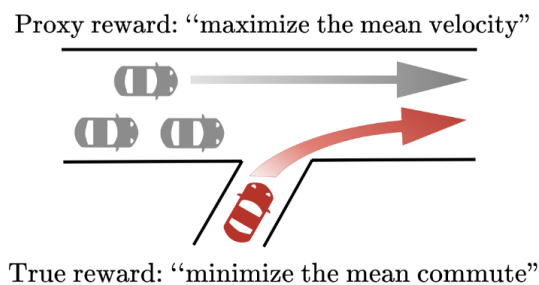


Figure 1: Illustration adapted from Pan, Bhatia, and Steinhardt (ICLR 2022).

- (a) Explain why parking can maximize the proxy reward even though it fails the intended objective. [8 pts]
- (b) Propose a revised reward or constraint that discourages this failure without directly using mean commute time. Explain one possible limitation of your proposal. [8 pts]

3 Formulating a real-world MDP (25 points)

Choose a real-world sequential decision problem, preferably one related to your interests. Describe the following components clearly:

- (a) the state space and what information a state must contain; [5 pts]
- (b) the action space and any important constraints on the actions; [5 pts]
- (c) the transition model, including the main sources of uncertainty; [5 pts]
- (d) the reward function and the behavior it is intended to encourage; and [5 pts]
- (e) at least one simplifying Markov assumption or modeling limitation. [5 pts]

You do not need to be precise in the description of the transition model and reward (no formula is needed). Qualitative description is enough.

4 Two-state policy evaluation (35 points)

Consider the continuing Markov reward process (no terminal state and interaction continues indefinitely) induced by a fixed policy π . The rewards are $r^\pi(x_1) = +1$ and $r^\pi(x_2) = -1$. The labeled arrows below give the non-self-loop transition probabilities; determine the self-loop probabilities using normalization. Let $\gamma = 0.9$ and use column vectors ordered as (x_1, x_2) .

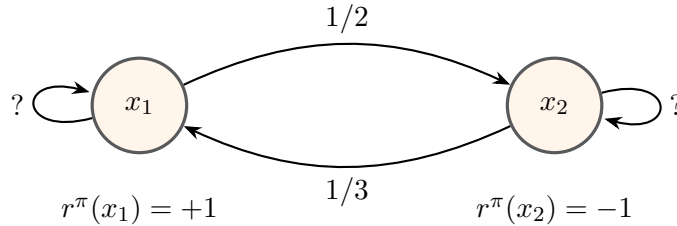


Figure 2: Two-state process for policy evaluation.

- (a) Write the complete transition matrix P^π . [5 pts]
- (b) Write the reward vector r^π . [5 pts]
- (c) Compute the truncated returns $V_T^\pi = \sum_{k=0}^{T-1} \gamma^k (P^\pi)^k r^\pi$ for $T \in \{2, 10, 50\}$. You may use a short program using MATLAB/Python for calculation, but report both state values for every T . [8 pts]
- (d) Write $V^\pi = r^\pi + \gamma P^\pi V^\pi$ explicitly as two scalar equations. [5 pts]
- (e) Solve those equations for the exact continuing-task value function V^π . Do not use value iteration. (Hint: solving a linear matrix equation.) [8 pts]
- (f) Compare V_T^π with V^π . Among the three tested horizons for $T \in \{2, 10, 50\}$, which is the smallest T for which $\|V_T^\pi - V^\pi\|_\infty < 0.1$? [4 pts]

Coding Assignment 1: Dynamic Programming (100 points)

RiverSwim MDP

You will implement planning algorithms for the six-state RiverSwim MDP. Action **LEFT** gives a small, easy reward $r = \frac{5}{1000}$ at the left boundary. Action **RIGHT** attempts to move against the current toward a larger reward at the right boundary $r = 1$. Stronger currents make rightward progress less likely. The supplied `RiverSwimMDP` stores the transition probabilities in $P[s, a, s_{\text{next}}]$ and expected immediate rewards in $R[s, a]$.

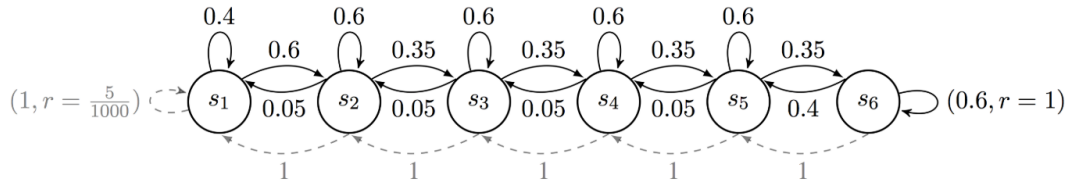


Figure 3: The RiverSwim MDP where dashed and solid arrows represent transitions for the **LEFT** and **RIGHT** actions, respectively. This figure is just an example. *The code uses a modified, customizable version* of what is shown in the figure where there are three different strengths (WEAK, MEDIUM, or STRONG) of the current (transition probabilities for being pushed back or successfully swimming RIGHT).

How to read the supplied code

The implementation uses integer indices for states and actions. These integers are labels used to access NumPy arrays; they are not rewards or probabilities.

- The RiverSwim environment is initialized using `mdp = RiverSwimMDP(current=current)`, where `current` specifies the current strength: "weak", "medium", or "strong".
- `LEFT = 0` and `RIGHT = 1` assign an integer index to each action. They are *action labels*, not complete policies. Thus, for example, `P[s, RIGHT, s_next]` accesses the probability of reaching `s_next` after choosing `RIGHT` in state `s`.
- The code numbers the six states 0, 1, ..., 5; these correspond to $s_1, s_2, \dots, s_6$ in the figure. For the provided RiverSwim environment, it is always `mdp.initial_state = 0`, the leftmost state (s_1 in the figure).
- `mdp.P[s, a, s_next]` is the transition probability $p(s' | s, a)$. For every state-action pair, summing this array over all possible next states gives one.
- `mdp.R[s, a]` is the expected immediate reward $r(s, a)$, whereas `value[s]` represents the current estimate of the long-term discounted value of state `s`.
- A deterministic `policy` is a one-dimensional integer array with one entry per state. The entry `policy[s]` is the selected action index: it is 0 for **LEFT** or 1 for **RIGHT**. For example, `[0, 0, 1, 1, 1, 1]` chooses **LEFT** in the first two states and **RIGHT** in the remaining four states.

- For action `RIGHT`, each current-strength tuple is ordered as $(p_{\text{back}}, p_{\text{stay}}, p_{\text{forward}})$. For example, `"weak": (0.05, 0.60, 0.35)` gives the three possible outcomes when the agent attempts to swim right from an interior state. At a boundary, probability mass for movement outside the state space remains at that boundary.
- `discount` is the discount factor γ ; `tolerance` controls the convergence stopping rule; and `max_iterations` prevents an infinite loop if an implementation does not converge.

Tasks

Complete only the marked TODO blocks in `coding/vi_and_pi.py`.

1. **(Coding)** Implement `bellman_backup`, which returns $Q(s, a)$ for one state-action pair. [15 pts]
2. **(Coding)** Implement iterative `policy_evaluation`. [20 pts]
3. **(Coding)** Implement `policy_improvement` and `policy_iteration`. [25 pts]
4. **(Coding)** Implement `value_iteration`. [25 pts]
5. **(Written)** Run `coding/run_experiments.py`. The code environment includes three current strengths—`WEAK`, `MEDIUM`, and `STRONG`—which determine the transition probabilities when the agent attempts to swim `RIGHT`. Each tuple specifies $(p_{\text{back}}, p_{\text{stay}}, p_{\text{forward}})$; for example, `"weak": (0.05, 0.60, 0.35)` means that the agent moves one state backward with probability 0.05, remains in the same state with probability 0.60, and moves one state forward with probability 0.35 (for example s_2 in the figure).

For each current strength:

- (a) Report the largest discount factor on the supplied grid

i.e., `DISCOUNT_GRID = np.round(np.arange(0.00, 1.00, 0.01), 2)`

for which the optimal action in the initial state is `LEFT`.

- (b) Compare the solutions produced by policy iteration and value iteration.
- (c) Describe and explain, both quantitatively and qualitatively, how the optimal values and threshold discount factors change as the current becomes stronger.

[15 points]

ECE 517 graduate extension. Within part (c), also evaluate at least 20 evenly spaced discount factors in $[0.80, 0.999]$, plot $V^*(s_0)$ versus γ for all three current strengths, and explain how current strength and discounting jointly affect the optimal behavior. This extension is assessed within the 15 points for part (c).

Required checks. Before submission, run `python -m pytest coding/test_public.py`. The public tests check shapes, tie-breaking, a small deterministic MDP, and agreement between policy and value iteration. Passing public tests does not guarantee full credit; additional valid MDPs and tolerances will be used for grading. Do not special-case the supplied environments.

Coding rubric

Component	Points
Bellman backup	15
Policy evaluation	20
Policy improvement and policy iteration	25
Value iteration	25
Experiments, interpretation, and reproducibility	15
Total	100

Implementation and submission checklist

- Use synchronous value updates: compute an entire new vector from the previous vector before replacing it.
- Compute Bellman expectations from `mdp.P` and `mdp.R`; do not hard-code RiverSwim states, rewards, policies, or thresholds.
- Return NumPy arrays with the required shapes and an integer dtype for policies.
- Check convergence with the infinity norm and honor `max_iterations`.
- Confirm that all public tests pass from a clean environment and that `run_experiments.py` completes without warnings.
- Open your final ZIP before uploading and verify that it contains the completed source files but no virtual environment or cache folders.